Hybrid Temporal Forecaster:

Residual Decomposition for Hourly Electricity Data (ETTh1)

Phase 1 — Hybrid Temporal Forecaster Project

March 22, 2026

Abstract

We present a hybrid forecasting pipeline for hourly electricity data that combines **SARIMAX** (trend and seasonality), **Gaussian Mixture** regime features on residuals, and a **simplified TFT-style** neural model trained on multi-step residuals, with an **additive** final forecast: $\hat{y} = \hat{y}^{\text{SARIMAX}} + \hat{r}^{\text{neural}}$. On the ETTh1 benchmark (17,420 hourly observations, target **OT**), we use a strict chronological train/validation/test split (60%/20%/20%). The statistical baseline achieves strong test metrics; the hybrid path does not improve sMAPE/MAE/RMSE in our run—an honest outcome on a smooth seasonal series where residuals after SARIMAX are small. The pipeline is reproducible with `python scripts/run_all.py`.

1 Introduction

1.1 Motivation

High-frequency energy and IoT series mix **repeatable structure** (e.g. daily cycles) with **time-varying volatility** and nonlinear errors. Pure classical models capture seasonality well but may miss structure in **residuals**; pure deep models may overfit or underuse clear seasonality. We follow **residual decomposition**: explain the dominant seasonal part first, then model what remains.

1.2 Problem

Forecast **OT** (oil temperature) on ETTh1 for horizon H=24 hours, using covariates **HUFL, HULL, MUFL, MULL** in the neural block, with **no** temporal leakage: train/val/test are **chronological**; features are **causal**.

1.3 Proposed pipeline

1. **SARIMAX** on OT: $(1, 0, 1) \times (1, 0, 1, 24)$.
2. **Residuals**: observed minus SARIMAX fitted/forecast as implemented.
3. **GMM** (K=2) on train-only causal residual features → regime probabilities.
4. **TFT-style** network predicts multi-step residuals (early stopping).
5. **Hybrid**: $\hat{y}_{t+h} = \hat{y}^{\text{SARIMAX}}_{t+h} + \hat{r}^{\text{neural}}_{t+h}$.

2 Related work

Statistical forecasting: ARIMA/SARIMAX and seasonal models [1]. **Deep sequence models:** LSTM [4]; TFT [2]; Informer [3]. **Hybrids:** forecast combination [7]; residual-style hybrids [8]. Tools: **statsmodels** [5], **scikit-learn** [6].

3 Dataset and splits

ETTh1 (ETT-small) is hourly, July 2016–June 2018 (∼17,420 rows). Target: **OT**; covariates for the neural block: HUFL/HULL/MUFL/MULL. Splits: **60% / 20% / 20%** in time—no shuffling (Figure 1). EDA (`01_eda.ipynb`) shows strong **24-hour** seasonality and varying volatility.

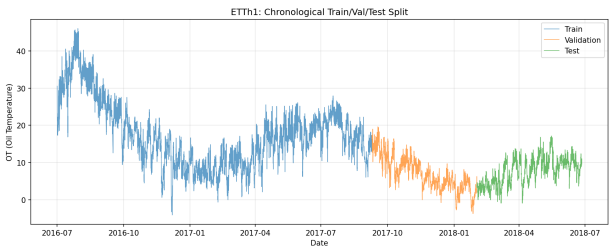

Figure 1: Chronological split of OT on ETTh1 (train / val / test).

4 Methodology

4.1 SARIMAX

Order $(1, 0, 1)$, seasonal $(1, 0, 1, 24)$ on OT (statsmodels). Outputs: fitted values, residuals, multi-step forecasts aligned with the neural head.

4.2 GMM regimes

Causal features from residuals (lag, rolling volatility, etc.); GMM **fit on train only**; `predict_proba` on val/test.

4.3 TFT-style residual model

Encoder–decoder style network in `models_tft.py`: MSE on residuals, Adam, dropout, gradient clipping, early stopping (Phase 1; simplified vs full pytorch-forecasting TFT).

4.4 Hybrid

$\hat{y} = \hat{y}^{\text{SARIMAX}} + \hat{r}^{\text{neural}}$ in `hybrid.py`—**additive** combination.

5 Experiments

5.1 Metrics

Primary **sMAPE**; also **MAE** and **RMSE**.

$$\text{sMAPE} = \frac{100}{n} \sum_i \frac{|y_i - \hat{y}_i|}{(|y_i| + |\hat{y}_i|)/2 + \varepsilon} \tag{1}$$

5.2 Results

Table 1: held-out **test** (as `run_all.py`).

Table 1: Test-set comparison (24-step horizon, as in pipeline).

Model	sMAPE (%)	MAE	RMSE
SARIMAX	116.93	6.09	7.07
Hybrid	120.16	6.17	7.15

On this run **SARIMAX is slightly better** on these aggregates: OT is smooth and seasonal; residuals after SARIMAX are small. Per-horizon CSV: `figures/per_horizon_results.csv`.

6 Discussion

Strengths: modular `src/`, one entry `run_all.py`, causal features, chronological evaluation. **Limitations:** fixed SARIMAX orders; simplified TFT; point forecasts only. **Phase 2:** auto orders, full TFT, uncertainty, multi-series.

7 Conclusion

We delivered SARIMAX + GMM + TFT-style residual + additive hybrid on ETTh1 with reproducible code and figures. Results are reported honestly: the linear seasonal baseline is very competitive here.

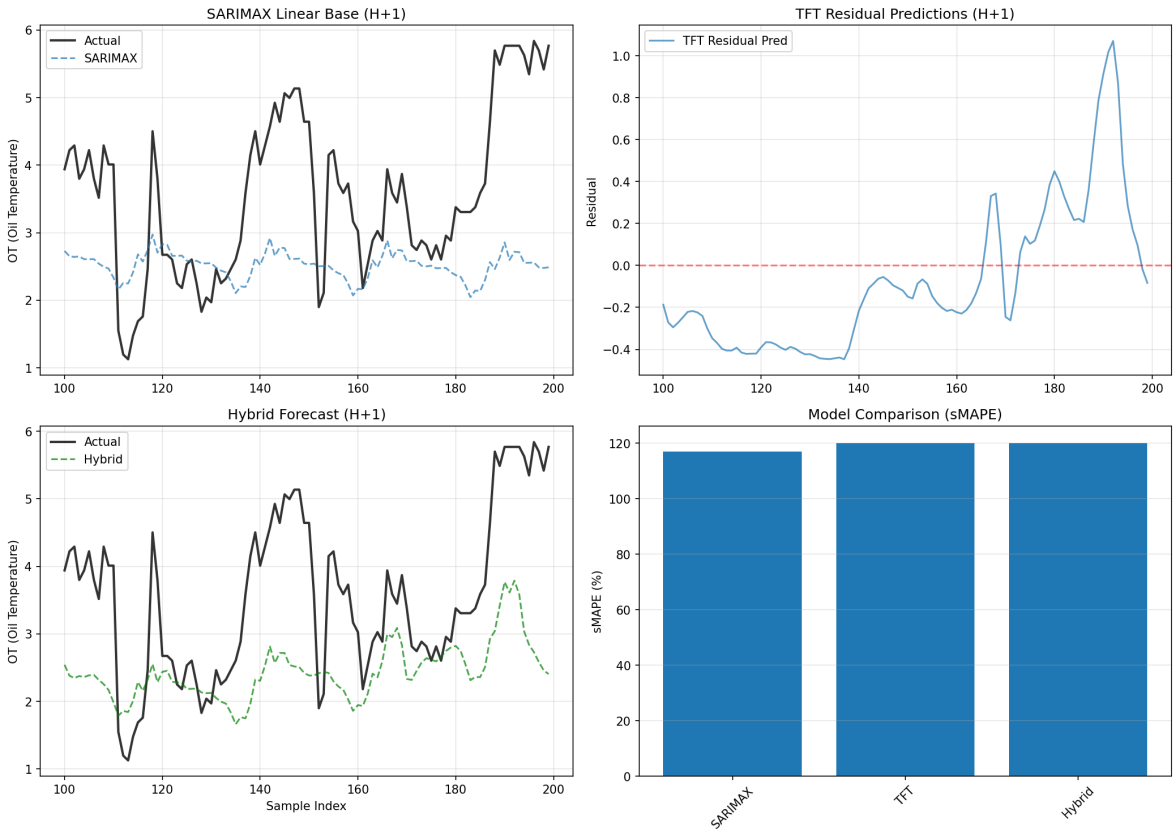

Figure 2: Model comparison plots (output of `run_all.py`).

References

[1] Hyndman, R.J., & Athanasopoulos, G. (2021). *Forecasting: Principles and Practice* (3rd ed.). OTexts.

[2] Lim, B., et al. (2021). Temporal Fusion Transformers for interpretable multi-horizon forecasting. *Int. J. Forecasting*, 37(4), 1748–1764.

[3] Zhou, H., et al. (2021). Informer: Beyond efficient transformer for long sequence time-series forecasting. *AAAI*.

[4] Hochreiter, S., & Schmidhuber, J. (1997). Long short-term memory. *Neural Computation*, 9(8), 1735–1780.

[5] Seabold, S., & Perktold, J. (2010). statsmodels: Econometric and statistical modeling with python. *SciPy*.

[6] Pedregosa, F., et al. (2011). Scikit-learn: Machine learning in Python. *JMLR*, 12, 2825–2830.

[7] Timmermann, A. (2006). Forecast combinations. *Handbook of Economic Forecasting*, 1, 135–196.

[8] Smyl, S. (2020). A hybrid method of exponential smoothing and recurrent neural networks for time series forecasting. *Int. J. Forecasting*, 36(1), 75–85.

Appendix A: Reproducibility

```
python3 -m venv venv
source venv/bin/activate
pip install -r requirements.txt
python scripts/download_data.py
python scripts/run_all.py
```

Outputs: `figures/*.png`, `figures/*.csv`.

Appendix B: Layout

`src/`: `data_prep`, `baselines`, `gmm_regimes`, `models_tft`, `hybrid`, `metrics`; `scripts/run_all.py`; `notebooks/01_eda.ipynb`.